COMP 018: Database Administration - Activity 6 Progress Report

Database Backup Planning, Verification, Restoration, and Disaster Recovery Simulation

Project Identification & Metadata

Course	COMP 018 - Database Administration
Evaluating Group	Group 4: Olivarez, Charles Kean E., Infantado, Gerwin G., Pocsidio, Jhon Louie F., Abangan, SJ Kaizen M., Bartolata, Redge Raindel N.
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	BoardTrack: Boarding House & Rental Management System
Target Project Group	Group 11: Barcarse, Adrian; Cataluña, Zyruz Gabriel; Camu, Edgar; Policarpio, Kurt; Crucillo, Karl Robert
Target Project Repository	https://github.com/karlrobertc-art/BoardTrack
Primary Database	boardtrack_db (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	boardtrack_db_restore_test

ACTIVITY 6: Database Backup, Restoration, and Recovery

3.1 Backup Planning & Architecture

The disaster recovery plan for Group 11's BoardTrack database encompasses eight relational tables, one view, and synchronization triggers within the public schema, capturing boarding houses, rental rooms, tenant profiles, lease contracts, rental payments, maintenance logs, user accounts, and roles. The plan covers sequence counters, primary keys, foreign keys, temporal exclusion constraints, triggers, and privilege grants to ensure operational continuity. The backup strategy utilizes PostgreSQL custom compressed archive format via a dedicated backup operator role (boardtrack_backup). Archive dumps are stored in a protected directory outside web roots and repository trees. Recovery objectives define a Recovery Point Objective of 1 hour and a Recovery Time Objective of 30 minutes. Retention guidelines specify maintaining seven daily backups alongside four weekly snapshots.

Because database login accounts exist at the PostgreSQL cluster level and are not stored within single-database dumps, disaster recovery plans require executing access-control role scripts on new server instances prior to restoring database contents.

3.2 Backup Creation, Checksum & Archive Inspection

Logical backup creation was executed using the dedicated backup account, producing a compressed custom format archive sized at 35,849 bytes. Cryptographic verification computed a SHA-256 hash value of 5B83883ADEB30784905A7EB22B7B67607DC6D1CCB22A508E5559DE60EB48559A, establishing a baseline for integrity verification.

Inspection of the archive table of contents verified 98 catalog entries, confirming the inclusion of all eight database tables, view v_lease_summary, sequence objects, primary keys, check constraints, temporal exclusion constraint ex_lease_no_overlap, foreign keys, triggers, and default privilege grants. To ensure recoverability beyond file inspection checks, a complete restoration drill was conducted in an isolated test database.

3.3 Restoration into Separate Test Database

An isolated test database named boardtrack_db_restore_test was created on the local server instance. The custom archive dump file was restored into this database using administrative utility tools. The restoration process completed cleanly with zero error codes.

3.4 Post-Restore Verification Suite

Post-restoration verification confirmed that all eight relational tables and view v_lease_summary were recreated with proper schema structure under administrative ownership. Record count validation across tables demonstrated complete parity with the source database, matching properties, rooms, tenant profiles, leases, payments, and maintenance tickets. Database checks confirmed that all check, trigger, and temporal exclusion constraints were fully restored.

Access control verification on the restored test database confirmed that privilege definitions transferred accurately. Queries executed under the application runtime account successfully retrieved active room records, while query attempts on landlord passwords in app_user by reporting accounts were blocked by the database security layer.

3.5 Controlled Recovery Simulation

A controlled data recovery simulation was performed to test incident response procedures. A disposable property record was inserted into the restored test database to establish a baseline state. A data loss event was then simulated by removing the record, and queries confirmed its deletion.

Targeted record restoration was executed to reinsert the missing data row without affecting existing database entries. Post-recovery verification confirmed that the record was fully restored with its original attributes intact. Following verification, the test record was removed to clean up the test environment.

3.6 Limitations, Evaluation & Production Recommendations

Restoration procedures on fresh server installations require pre-executing role definition scripts to establish user accounts before restoring database ACLs. The PostgreSQL extension btree_gist must be enabled on the target server prior to restore to support the temporal exclusion constraint. For production operations, daily automated backup generation should be scheduled using system task schedulers to archive database dumps with date-stamped filenames and retain snapshots according to policy. Enabling continuous write-ahead log archiving will support point-in-time recovery capabilities.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 6)

Column Header	Value
Project	Group 11 - BoardTrack: Boarding House & Rental Management System
Database and backup file	boardtrack_db, boardtrack_db_activity6.dump, Size: 35,849 bytes, SHA256: 5B83883ADEB30784905A7EB22B7B67607DC6D1CCB22A508E5559DE60EB48559A
Backup method	pg_dump custom format (-Fc), compressed binary dump (2026-11-05 18:48:31)
Restore test result	Success (Restored into boardtrack_db_restore_test; all 8 tables, constraints, triggers, and data verified)
Access control after restore	Yes (Activity 4 roles and permissions validated; boardtrack_report_user denied reading app_user)
Recovery simulation result	Recovered (Simulated deleted test property successfully restored from verified baseline state)
Proposed RPO and RTO	RPO: 1 hour, RTO: 30 minutes
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-6/Evidence-act6
Feedback sent to owners	Yes (05/11/2026)
PDF progress document	https://github.com/Nycri/brewpos/blob/main/PDF_PROGRESS_DOCUMENTATION.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.